DDLaunch+ — Data-Driven Launcher

Version 2.3.1 · Glenn Madine · Released 11/06/2026 Win32 C++ (Windows SDK) · nlohmann/json 3.12.0

DDLaunch+ is a small Windows launcher for the machines and resources you connect to every day. It shows a list of resources (hosts, drives or shell items), each with a **type** such as `RDP`, `SSH` or `HTTPS`. Double-clicking an entry runs the program or URL handler defined for that type.

The app has no built-in protocols. Both the list and the launch rules come from two JSON files that you edit in Notepad, and changes appear as soon as you switch back to the window.

Contents

1. Features
2. Requirements
3. Building
4. First run and the data folder
5. Configuration files
6. Using DDLaunch+
7. How a launch works
8. Messages and troubleshooting
9. Code walkthrough
10. Design notes
11. Known limitations
12. Version history

Features

- **Data-driven.** Resources come from `connections.json` and launch rules from `actionDefinitions.json`. Adding a new protocol is a JSON edit, not a code change.
- **Programs or URLs.** An action's command is either an executable (run with arguments) or a URL prefix of any scheme: `https://`, `ssh://`, `file:///`, `mailto:`, `ms-settings:` and so on.
- **Live reload.** Edits are picked up when the window is next activated. Only files whose timestamp or size changed are re-read.
- **Error-tolerant.** If a file is invalid, the error is shown once and the last good data stays loaded.
- **Clear error messages** for a missing type, an unknown type, an empty command, a failed launch (with the Win32 error text) and duplicate action names.
- **Sortable columns.** Sorting ignores case, and the sort order and selected row are kept across reloads.
- **Keyboard support.** Tab moves between controls and Enter launches. Esc does nothing, so it can't close the launcher by accident.
- **Per-Monitor V2 DPI awareness.** Text and layout stay sharp, and rescale when the window moves to a monitor with different scaling.

- **Remembers its window.** Position, size and maximized state are restored at the next start, and the window is kept on screen.
- **Single instance.** Starting it again brings the running copy to the front.
- **Run dialog** (the same one as Win+R) and an **ad-hoc Remote Desktop** button for hosts that aren't in the list.

Requirements

Item	Detail
OS	Windows 10/11. Starts on older versions with reduced DPI support.
Compiler	Microsoft C++ (developed with 19.51), `/std:c++17`
SDK	Windows SDK (`windows.h`, `commctrl.h`, `shellapi.h`, `shlwapi.h`, `shldisp.h`, `shlobj.h`)
Library	nlohmann/json single header, shipped as `json.hpp`
Libraries linked	`comctl32`, `shell32`, `shlwapi`, `user32`, `gdi32`, `ole32`, `oleaut32`

Building

Build files

File	Purpose
`DDLaunch.cpp`	The whole application
`DDLaunch.rc`	Resources: icon, application manifest (RT_MANIFEST id 1), version info (2.3.1.0)
`DDLaunch.manifest`	Common Controls v6, Per-Monitor V2 DPI, `asInvoker`, supported Windows versions
`resource.h`	Defines `IDI_ICON1` (101)
`DDLaunch.ico`	Application icon
`json.hpp`	nlohmann/json 3.12.0

Command line (Developer Command Prompt)

```
rc /nologo DDLaunch.rc
CL /EHsc /W3 /O2 /GL /DUNICODE /D_UNICODE /DNDEBUG /std:c++17 DDLaunch.cpp DDLaunch.res /Fe:DDL
aunch.exe /link /SUBSYSTEM:WINDOWS /MANIFEST:NO comctl32.lib shell32.lib shlwapi.lib user32.lib
gdi32.lib ole32.lib oleaut32.lib
```

Visual Studio project

- Add `DDLaunch.rc` to the project so the manifest and icon are embedded.
- Set **Linker › Manifest File › Generate Manifest = No**. The manifest must come only from `DDLaunch.rc`. If the linker adds a second one, the link fails with CVT1100 / LNK1123.

- Set **C/C++ › Language › C++ Language Standard = ISO C++17** (or later).
- Use Character Set = Unicode. The source defines `UNICODE` / `_UNICODE` only if they aren't already defined.

Build self-check. At startup the app checks that Common Controls v6 is loaded. If it isn't, the manifest wasn't embedded: a message goes to the debugger output, and debug builds also show a message box. Without the manifest the app still runs, but without visual styles or column sort arrows.

First run and the data folder

All user data lives in `%USERPROFILE%\DDLaunch\` (for example `C:\Users\glenn\DDLaunch`).

File	Created automatically?	Purpose
`actionDefinitions.json`	Yes, with RDP / SSH / HTTPS / HTTP	Launch rules, one per type
`connections.json`	Yes, as `[]`	The resources shown in the list
`settings.json`	Written on exit	Window position, size and maximized state

- The folder and the two config files are created whenever they're missing: at startup, on reload, and before opening a file for editing. Existing files are never overwritten.
- If `%USERPROFILE%` isn't set, the app uses the Profile known folder instead. If that can't be found either, it uses the folder containing `DDLaunch.exe`.
- All files are **UTF-8** (with or without a BOM). Notepad's default UTF-8 encoding is correct.

Configuration files

`actionDefinitions.json`

This file must be a **top-level JSON array**. Each entry needs all three string fields:

Field	Meaning
`action`	The type name. It is matched against a connection's `type`, ignoring case (`rdp`, `Rdp` and `RDP` are the same).
`command`	An executable path, or a URL prefix (see How a launch works).
`args`	For an executable: text placed **directly** in front of the host. Include any space you need, e.g. `" /v:"`. Ignored for URLs.

Default content:

```
[
    { "action": "RDP",   "args": " /v:", "command": "C:\\Windows\\System32\\mstsc.exe" },
    { "action": "SSH",   "args": " ",    "command": "C:\\Windows\\System32\\OpenSSH\\ssh.exe"
},
    { "action": "HTTPS", "args": "",     "command": "https://" },
    { "action": "HTTP",  "args": "",     "command": "http://" }
]
```

More examples:

```
    { "action": "SHARE",    "args": "",  "command": "file:///" },
    { "action": "SETTINGS", "args": "",  "command": "ms-settings:" },
    { "action": "PS",       "args": " -NoExit -Command Enter-PSSession -ComputerName ",
                           "command": "powershell.exe" }
```

Rules:

- Backslashes in JSON strings must be doubled (`C:\\Windows\\...`).
- If any entry is missing a field, or a field isn't a string, the **whole file is rejected**. An error is shown and the previously loaded actions are kept.
- **Duplicate names** (ignoring case) are reported once each time the file loads. Only the first entry with a given name is used.

`connections.json`

This file can be a **flat array**:

```
[
    { "host": "dc01.ynext.corp",   "type": "RDP",   "description": "Domain controller" },
    { "host": "admin@pi01",        "type": "SSH",   "description": "Raspberry Pi 01" },
    { "host": "piweb.ynext.corp",  "type": "HTTPS", "description": "Internal web server" }
]
```

...or an **object of named groups**, where each value is one entry or an array of entries:

```
{
    "Servers": [
        { "host": "dc01.ynext.corp", "type": "RDP", "description": "Domain controller" }
    ],
    "Web": { "host": "piweb.ynext.corp", "type": "HTTPS", "description": "Internal web server"
}
}
```

Field	Meaning
`host`	What gets launched: a host name, `user@host`, `host:port`, a path, or a full URL
`type`	Which action to use
`description`	Free text shown in the list

Notes:

- All fields are optional strings. A missing or non-string field is shown as blank. Other fields are ignored.
- Group names organize the file but aren't shown in the list.
- If the file can't be read or isn't valid JSON, an error is shown once and the current list is kept until the file is fixed.

`settings.json`

Written automatically when the app closes (and when Windows logs off or shuts down):

```
{
    "window": { "x": 120, "y": 80, "width": 640, "height": 420, "maximized": false }
}
```

- `x` / `y` are screen pixels. `width` / `height` are in 96-DPI units, so the window keeps the same apparent size on monitors with different scaling.
- The file is written to a temporary file first and then swapped in, so it can't be left half-written.
- Values that are out of range (under 100 or over 20000) or a corrupt file are ignored silently, and the default size is used.

Using DDLaunch+

```
+-------------------------------------------------------------------------+
| DDLaunch+ v2.3.1 - double-click a row or press Enter to launch          |
+-------------------------------------------------------------------------+
| Resource (Host, Drive, or Shell Item) |  Type  | Description           |
| dc01.ynext.corp                       |  RDP   | Domain controller     |
| admin@pi01                            |  SSH   | Raspberry Pi 01       |
| ...                                                                     |
|                                                                         |
| [ Launch RDP connection for host...] [ Edit actionDefinitions.json ] |
| [ Edit connections.json           ] [ Run...      ] [ Exit        ]  |
+-------------------------------------------------------------------------+
```

Control	Action
Double-click a row / **Enter** on the list	Launch the selected entry
Click a column header	Sort by that column; click again to reverse the order
Launch RDP connection for host not in this list	Opens Remote Desktop Connection (`mstsc.exe`) empty
Edit actionDefinitions.json / **Edit connections.json**	Opens the file in Notepad
Run...	The Windows Run dialog (same as Win+R)
Exit	Saves the window position and closes

Keyboard

- Tab and Shift+Tab move between the list and the buttons, in the order shown on screen.

- Enter or Space presses the focused button. Enter on the list launches the selected entry.
- **Esc is ignored** on purpose.
- When you switch back to the window, focus returns to the control that last had it.

Live editing. Edit a file in Notepad, save it, and click back into DDLaunch+. The list updates and the selected row and sort order are kept.

Window. There's a minimum size, so the buttons never overlap and some of the list always shows. If a shortcut is set to "Run: Minimized", that setting takes priority over a saved maximized state.

How a launch works

1. The selected row's ID is used to look up the entry (not the row position or the cell text).
2. Its `type` is matched to an action, ignoring case.
3. The action's `command` decides how the launch is done:

Executable command

Any command without a URL scheme (`C:\...`, `ssh.exe`, `\\server\share\tool.exe`) is run as:

```
command   args + host
```

If the host contains spaces, tabs or quotes, it is quoted using standard Windows command-line rules, so it arrives as one argument. For example, with `args` = `" /v:"` and host `Lab PC 7`, the parameters are `/v:"Lab PC 7"`. Plain hosts are passed unchanged.

URL command

A command starting with an RFC 3986 scheme of two or more characters is treated as a URL (`C:` is a drive letter, not a scheme). `args` is ignored.

Host in `connections.json`	Command	Opened
`piweb.ynext.corp`	`https://`	`https://piweb.ynext.corp`
`dc01:8443`	`https://`	`https://dc01:8443` (host:port is still appended)
`https://piweb/x?a=1#f`	`https://`	`https://piweb/x?a=1#f` (already a full URL; used as-is)
`http://old`	`https://`	`http://old` (full URL with a different scheme; used as-is)
`a@b` / `mailto:a@b`	`mailto:`	`mailto:a@b`
`server/My Page`	`https://`	`https://server/My%20Page`
`C:\Temp\a b.txt`	`file:///`	`file:///C:\Temp\a%20b.txt`

- A host is treated as a full URL if it has a scheme followed by `//`, or if it uses the action's own scheme.

- Characters that are never valid in a URL (spaces, control characters, and `" < > ^ ` { | }`) are percent-encoded.
- `/ ? # : % \` and non-ASCII text are left as they are.

Every launch goes through `ShellExecuteExW`. If Windows can't open it, DDLaunch+ shows a single message with the command, the parameters and the Win32 error. Declining a UAC prompt is not treated as an error.

Messages and troubleshooting

Message	Cause / fix
No type	The entry has no `type`. Add one in `connections.json`.
Unknown type	No action matches. The message lists the defined actions; add one, or fix the type.
No command	The matching action has `"command": ""`.
Launch failed (Error *n*)	Windows couldn't run it: wrong path, no handler for the URL scheme, etc. Check `command` and `args`.
Duplicate actions	Two actions have the same name (ignoring case). Only the first is used.
connections.json is not valid JSON	Shows the parser's message with its position. The old list is kept.
Could not load actionDefinitions.json	Invalid JSON, not an array, or an entry missing `action` / `args` / `command`. The old actions are kept.
Could not create the DDLaunch data folder	No permission in the profile folder, or the path is invalid.
Common Controls … is loaded instead of 6.0 (debug build)	The manifest isn't embedded. Build `DDLaunch.rc`, link `DDLaunch.res`, and turn off the linker's own manifest.

- Each error is shown **once per save** of the file. Fix the file and save it again to reload it.
- A file saved by Notepad as "UTF-16" or "ANSI" with non-ASCII characters won't parse. Save it as **UTF-8**.

Code walkthrough

`DDLaunch.cpp` is organized from the bottom up, in this order:

Area	Key functions / types	What it does
Text conversion	`Utf8ToWide`, `WideToUtf8`, `ExceptionText`	Converts between UTF-8 (on disk and in nlohmann/json) and UTF-16 (Win32). Exception text is converted before it reaches `MessageBoxW`.

Area	Key functions / types	What it does
Actions	`struct Action`, `from_json`, `loadActions`, `findAction`	Parses `actionDefinitions.json` and finds actions ignoring case (`CompareStringOrdinal`).
File reading	`ReadFileUtf8`	Reads with `CreateFileW` / `ReadFile`, so non-ASCII paths work. Allows read, write and delete sharing.
Devices	`struct Device`, `LoadConnections`	Accepts an array or an object of groups. Returns `std::nullopt` on error so the caller keeps the old list.
Paths and seeding	`GetUserProfileDir`, `GetDataDir`, `EnsureDataFiles`, `CreateFileWithDefaultContent`	Resolves `%USERPROFILE%\DDLaunch` and creates missing files (`CREATE_NEW`, never overwrites).
Editing	`OpenJsonFileForEditing`	Runs `notepad.exe "<path>"` directly, so it doesn't depend on the `.json` file association.
Launching	`StartsWithUrlScheme`, `HostIsFullUrl`, `EncodeUrlUnsafeChars`, `QuoteArgIfNeeded`, `LaunchSomething`, `ShellOpen`, `ReportLaunchFailure`, `GetErrorText`	Detects URLs, builds the URL or quotes the argument, launches via `ShellExecuteExW` and reports errors.
Run dialog	`ShowRunDialog`	`Shell.Application` → `IShellDispatch::FileRun` (documented API, not the shell32 ordinal 61).
DPI	`namespace Dpi` (`EnableDpiAwareness`, `ForWindow`, `ForMonitor`, `Scale`, `CreateUiFont`, `WindowSizeForClient`)	Newer APIs are loaded at run time, so the exe still starts on older Windows.
Placement	`ClampToWorkArea`, `WindowSettings`, `LoadWindowSettings`, `SaveWindowSettings`, `TrackNormalRect`	Saves and restores the window, and keeps it inside the monitor's work area.
Sorting	`ApplyCurrentSort`, `ToggleSortColumn`, `UpdateHeaderSortArrow`	`std::stable_sort` comparing with `CompareStringOrdinal` (ignoring case), plus the header arrows.
Identity	`AssignDeviceIds`, `FindDeviceIndexById`, `GetRowDeviceId`	Stable IDs stored in each row's `lParam`.
List view	`PopulateListView`	Refills the list with redraw paused, restoring the selection and focus.
Reload	`FileStamp`, `GetFileStamp`, `ReloadAllData`, `WarnAboutDuplicateActions`	Reloads only changed files, with a re-entrancy guard and one error per save.
Launch logic	`OnItemActivated`, `ActivateSelection`	Validates the entry, finds the action, launches and reports.

Area	Key functions / types	What it does
Layout	`namespace Layout`, `S()`, `MeasureButtonWidth`, `ApplyDpi`, `RescaleForDpi`	96-DPI design values. Buttons are sized to fit their text.
Window	`WndProc`	`WM_CREATE`, `WM_SIZE`, `WM_GETMINMAXINFO`, `WM_MOVE`, `WM_DPICHANGED`, `WM_COMMAND`, `WM_NOTIFY`, `WM_ACTIVATE`, `WM_SETFOCUS`, `WM_ENDSESSION`, `WM_DESTROY`
Startup	`CheckCommonControlsVersion`, `AcquireSingleInstance`, `wWinMain`	Single-instance check → COM (STA) → DPI → common controls → window class → placement → show → first load → message loop.

Startup sequence (`wWinMain`)

1. **Single instance.** If the `Local\DDLaunchPlus.SingleInstance…` mutex already exists, find the `DDLaunchClass` window, restore it, bring it to the front and exit.
2. `CoInitializeEx` (single-threaded apartment, for `ShellExecuteEx` and the Run dialog).
3. DPI awareness fallback (normally already set by the manifest), then `InitCommonControlsEx` and the Common Controls v6 check.
4. Register the class (large and small icons at their exact sizes) and create the hidden window. `WM_CREATE` builds the list view and buttons and applies the DPI font.
5. Work out the window rectangle from `settings.json` or the default, scaled for the target monitor and clamped to its work area. Move the window with `g_placingWindow` set so it isn't scaled twice, then rescale to the final DPI.
6. Show the window (maximized if it was saved that way), then do the **first data load**, so any error box appears over a visible window.
7. Message loop with `IsDialogMessageW`, then `CoUninitialize`.

Design notes

- **Change detection.** A `FileStamp` (exists, last-write time, size) is recorded before parsing. On activation, a file is re-read only if its stamp changed. This prevents an error box from reappearing in a loop, and avoids reading the files again on every focus change.
- **Re-entrancy.** `MessageBoxW` runs its own message loop, which can deliver `WM_ACTIVATE` in the middle of a reload. `g_inReload` (cleared automatically on every exit path) ignores those nested calls.
- **Stable device IDs.** On reload, IDs are carried over first by an exact match (host + type + description), then by host + type. So the selection survives sorting and editing a description, and launching never depends on the row position.
- **Case rules.** Action matching, duplicate detection and sorting all use `CompareStringOrdinal(…, TRUE)`, the same invariant rule the file system uses, so the result doesn't depend on locale.
- **DPI.** All sizes are 96-DPI design values passed through `S()`. On `WM_DPICHANGED` the font, column widths and button widths are rebuilt, and the window takes the size Windows suggests.

- **Single manifest.** The manifest comes only from `DDLaunch.rc`. There is no `#pragma comment(linker, "/manifestdependency…")`, and the linker is built with `/MANIFEST:NO`.
- **Single instance per logon session.** The mutex is in the `Local\` namespace, so separate RDP sessions each get their own copy.

Known limitations

- Group names in `connections.json` aren't shown, and the list can't be filtered or searched.
- The sort column isn't saved between runs; the list opens in file order.
- **Edit** buttons always use Notepad.
- `args` is ignored for URL actions, and a URL can only be built as prefix + host.
- Only the characters listed above are percent-encoded. A literal `#` or `%` in a host is passed as-is.
- The **Launch RDP…** button runs `mstsc.exe` found through the search path and the App Paths registry key.

Version history

Version	Highlights
2.3.1	Version resource corrected; `resource.h` cleaned up; COM initialized once on the UI thread; ordinal sort; config files opened with full sharing; URL hosts may be full URLs, and unsafe characters are percent-encoded; first load happens after the window is shown.
2.3.0	Hosts with spaces are quoted; duplicate action warning; stable device IDs; single instance; new window title.
2.2.0	Action names ignore case; selection kept across sort and reload; a bad `connections.json` keeps the old list; Unicode-safe error messages; dead code removed.
2.1.2	One manifest source (`DDLaunch.rc` + `/MANIFEST:NO`); Common Controls v6 self-check.
2.1.1	Window position saved and restored; window kept on screen; `UNICODE` guards.
2.1.0	Keyboard navigation; visual styles; Per-Monitor V2 DPI; buttons sized to fit their text.
2.0.3	Clear launch error messages via `ShellExecuteExW`.
2.0.2	URLs of any scheme detected; `file:///` fixed.
2.0.1	Reload only on change (fixes an error loop); sort kept across reloads.